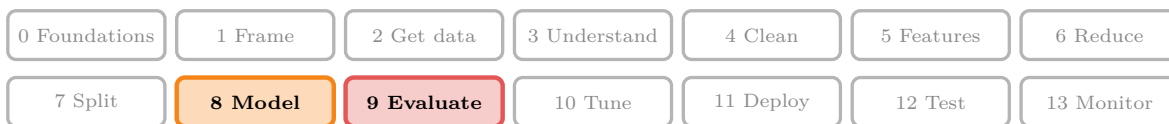


Why Random Forests Work: Bias and Variance

Contents

1. Overview	2
2. Why a forest lowers the variance	3
3. Seeing it in classification	3
4. Seeing it in regression	4
5. Summary	6
6. Sources	6
7. Key terms	6

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the [Course map](#).



- **Builds on:** Polynomial regression ([Note 61](#)); Overfitting ([Note 91](#)); Underfitting ([Note 91](#)); Hyperparameter tuning ([Note 98](#)); Decision trees ([Note 100](#)); Feature importance ([Note 100](#)).
- **Leads to:** Boosting ([Note 119](#)); Balanced random forest ([Note 133](#)).
- **Compare with:** K-nearest neighbours ([Note 91](#)); Bagging ([Note 107](#)); Dropout ([Note 1024](#)).

1. Overview

Key point: A random forest takes fully grown trees, which have low bias and high variance, and averages away most of the variance. The result has low bias and low variance, the combination single models rarely reach.

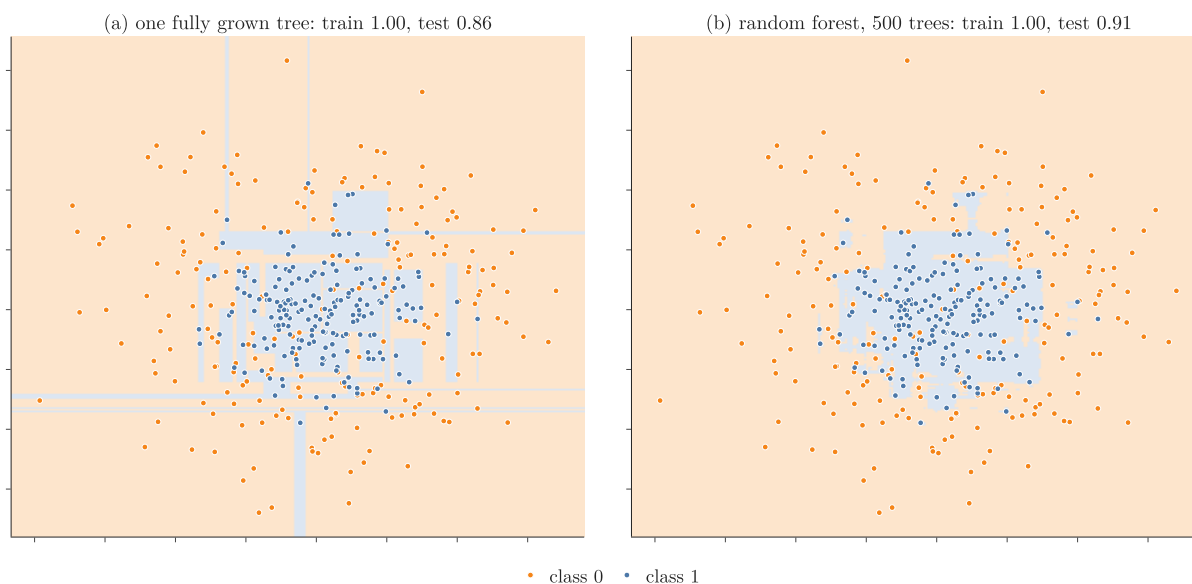


Figure 1: Concentric circles: (a) one fully grown tree draws boxes around single points; (b) a random forest of 500 trees draws one smooth inner region. Titles give the accuracy on the training and test sets

Figure 1 shows the whole story. One fully grown tree (a) carves out thin strips and small boxes for single points: it overfits. A forest of 500 such trees (b) keeps the shape of the data, a blue disc inside an orange ring, and drops most of the strips. Its test accuracy rises from 0.86 to 0.91.

This Note explains why, using the bias-variance trade-off, and shows the same effect on a regression problem. The Notebook (`notebook.ipynb`) computes every number; `images/figs.py` draws the figures.

2. Why a forest lowers the variance

Key point: A random forest is bagging with fully grown trees: each tree keeps the bias low, and averaging many of them removes most of the variance.

We want low bias and low variance, but a single model usually trades one for the other (the [bias-variance Note](#)). Bagging escapes this by averaging low-bias, high-variance models, each trained on its own random sample, so a change in the data is spread across many models (the [bagging Note](#), section 3.2). Think of many people guessing the number of sweets in a jar: each guess is off, some high and some low, but the average of the guesses lands close to the truth.

A random forest applies this to **fully grown** trees (no `max_depth`). Each tree fits its training data almost perfectly, so the forest starts with low bias; averaging hundreds of them keeps the bias low and cuts the variance. The next two sections measure the effect.

3. Seeing it in classification

Key point: On noisy concentric circles, one tree scores 0.86 on the test set and a forest of 500 trees 0.91.

The data is made with `make_circles` (the [kernel trick code Note](#)): 500 **observations** (records, one row of the data table each), 2 **features** (input variables, one column each) and a **target** (the output we predict) with two classes. The blue class (1) sits in a small disc inside the orange class (0), with plenty of noise, so the classes overlap. We train on 400 points and test on 100.

One fully grown tree (Figure 1a). Its surface has long thin strips and isolated boxes. Each exists because of a single point: a lone blue point among orange ones gets its own blue strip, although the region around it clearly belongs to orange.

- Training accuracy: **1.00**, every training point is classified correctly: low bias.
- Test accuracy: **0.86**. A new sample from the same pattern would produce different strips in different places: high variance. This is overfitting.

A random forest of 500 trees (Figure 1b). Most strips and boxes are gone. The surface captures the true nature of the data: a blue region inside an orange one.

- Training accuracy: still **1.00**.
- Test accuracy: **0.91**: the variance has dropped.

Python: The two models.

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
rf = RandomForestClassifier(n_estimators=500,
                           random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
rf.score(X_test, y_test)    # 0.91
```

`n_estimators` is the number of trees and `n_jobs=-1` trains them on every CPU core (both from the [bagging classifier Note](#)). The surfaces are drawn on a grid of points, as in the [KNN Note](#), section 5.

Extra: One might expect the forest to give up a little training accuracy in exchange for the lower variance. Here it does not. Each tree's bootstrap sample holds about 63% of the training observations (ESL §7.11), and a fully grown tree gets every observation it saw right. So for any training observation, the trees that saw it are a majority, and they outvote the rest.

The Notebook checks this: every training observation was seen by at least 57% of the 500 trees, and those trees were always right on it. Raising the noise of `make_circles` up to 1.0 leaves the forest's training accuracy at 1.00.

4. Seeing it in regression

Key point: On a noisy curve, one tree chases every point (test MSE 0.0192); a forest of 1,000 trees stays closer to the true pattern (test MSE 0.0140), exactly like bagging, since one feature leaves no features to sample.

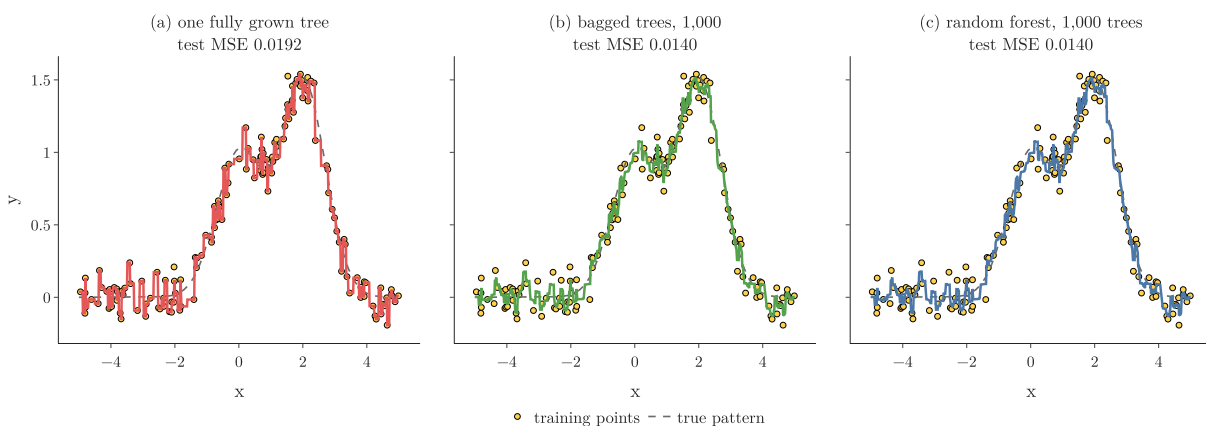


Figure 2: Two bumps plus noise: (a) one fully grown regression tree; (b) bagging with 1,000 fully grown trees; (c) a random forest of 1,000 trees. The dashed curve is the true pattern

The data is the two-bumps curve of the [bagging regressor Note](#), which compares one tree

with bagging. Here we add the random forest. We train on 150 points and test on 1,000; the dashed curve in Figure 2 is the true pattern.

- **(a) One fully grown tree** (red) passes through every training point: training error 0, test **mean squared error (MSE)** (the [regression metrics Note](#)) **0.0192**.
- **(b) Bagging with 1,000 fully grown trees** (green) no longer reaches every outlier and stays closer to the dashed curve: test MSE **0.0140**.
- **(c) A random forest of 1,000 trees** (blue) draws almost the same curve, with the same test MSE. Its training MSE rises a little, from 0 to 0.0018, while its test MSE falls about 27% below the single tree's.

Panels (b) and (c) match because the data has a single feature. A random forest differs from bagging only in sampling features at each split (the [bagging vs random forest Note](#)); with one feature there is nothing to sample, and `RandomForestRegressor` uses every feature at every split by default anyway. The forest's extra gain shows only on data with many features.

Extra: Older code often prints `np.sum((y_test - pred) ** 2)` and calls it MSE. The printed number is the **sum** of squared errors (19.2 and 14.0 here, over 1,000 test points); the mean divides by the number of points. The comparison between models comes out the same, but only the mean can be compared across test sets of different sizes.

5. Summary

	One fully grown tree	Random forest
Bias	low	low
Variance	high	low
Circles: train / test accuracy	1.00 / 0.86	1.00 / 0.91
Curve: train / test MSE	0.0000 / 0.0192	0.0018 / 0.0140

- We want low bias and low variance, but single models usually trade one for the other.
- Fully grown trees are low bias, high variance: they overfit.
- A random forest averages many such trees, each trained on a different random sample, so noisy observations are spread out and their effect averages away.
- The result keeps the low bias and cuts the variance: smoother boundaries and curves, better scores on new data.

6. Sources

- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer. §7.11 (the bootstrap holds about 63.2% of distinct observations).

7. Key terms

No new terms. Bias and variance are defined in the [bias-variance Note](#); low-bias, high-variance base models in the [bagging Note](#), section 3.